

Linux 命令行 ELF 控制流图 (CFG) 绘制工具

—— 完全自研版本 (native)

《软件安全原理》课堂练习 · C++ 实现，不使用 STL 与任何第三方库

核心特点：ELF 解析、函数定位、x86-64 机器码解码、基本块划分、CFG 构造 **全部由自身代码实现**，运行时不启动任何外部程序

规模：49 个源文件，5992 行，6 层架构 **环境：**Kali GNU/Linux (x86-64) gcc 13.2.0

编译：手工 gcc 命令，0 错误 0 警告

一、任务要求与本方案

1.1 题目要求

要求项	具体内容
平台 / 形式	Linux 平台命令行程序
语言 / 限制	C++； 不使用 STL ，不使用其他第三方库
输入	ELF 格式的二进制代码文件， 可指定函数
输出	Graphviz 格式的图形数据
必备模块	反汇编、基本块识别、基本块执行关系识别、图形输出
设计要求	面向对象思想，使用 继承 和 多态

1.2 本方案：从"借助外部工具"到"完全自研"

环节	早期版本	本版本 (native)
文件读取	objdump 代劳	open / fstat / mmap 自己映射
ELF 解析	objdump 代劳	自己解析 文件头、节表、字符串表、符号表
函数定位	objdump --disassemble=名字	自己查 符号表，st_value → 文件偏移 → 机器码指针
指令解码	解析 objdump 的文本输出	自己从字节解码 ：前缀/REX/opcode/ModRM/SIB/dispatch/imm
基本块 / CFG / DOT	自己实现	自己实现（ 算法与结果保持一致 ）

验收确认：无 STL 头文件 · 无 Capstone / LLVM / libelf / libbfd · 无 **popen / system / exec / fork** · 源码中无 objdump / readelf / nm 调用

外部工具仅在开发阶段作为对照 **oracle** 使用（见第十一节），程序运行时不依赖。

1.3 数据流总览

```
ELF 文件
| open / fstat / mmap                                ① core  BinaryFile
▼
BinaryFile (映射区 data / size)
| magic / ELFCLASS64 / 小端 / EM_X86_64              ② elf  ElfHeaderView
▼
ElfHeaderView → SectionTable → StringTable
|                                                    ② elf  SymbolTable
▼
SymbolTable → FunctionTable → FunctionInfo (名字/地址/大小/机器码指针)
|                                                    ③ arch  X86_64Decoder
▼
Instruction 双向链表 (前缀→REX→opcode→ModRM→SIB→disp→imm)
| Leader 法                                          ④ cfg  BasicBlockBuilder
▼
BasicBlock 链表 → Edge 链表                          ④ cfg  EdgeAnalyzer
▼
ControlFlowGraph
| 可达性 / 循环提示 / 统计                          ⑤ analysis
▼
GraphWriter → DOT 文本 — (程序外部) dot -Tpng → 图片 ⑥ output
```

► 终端同步演示 `cd /home/kali/elfcfg && ls` , 让老师看到 `src / tests / docs_native` 三个目录

二、待分析的测试程序 tests/demo.c

为便于课堂按难度选择，测试程序包含 6 个函数，覆盖从"无分支"到"多分支+循环"的各种形态：

函数	结构特征	用途
simple_if	单条件分支	CFG 最简单的示例
nested_if	嵌套条件	多层分叉与汇合
loop_sum	for + while	两条循环回边
classify	if / else if / else + for	课堂主用例
switch_test	switch (GCC 生成跳转表)	演示已知限制
call_test	三次函数调用，无分支	观察 call 的处理

```
int classify(int x)
{
    int sum = 0;
    int i;

    if (x > 100) {                /* 分支 1 */
        sum = 100;
    } else if (x > 0) {            /* 分支 2 */
        sum = x;
    } else {                      /* 分支 3 */
        sum = -x;
    }

    for (i = 0; i < 3; i++) {      /* for 循环：形成向后跳转边 */
        sum = sum + i;
    }

    return sum;
}
```

► 终端同步演示 gcc -O0 -fno-pie -no-pie tests/demo.c -o demo 然后 ./demo 7 看运行结果

编译选项：-O0 关闭大部分优化，使控制流结构更容易与源码对应；-fno-pie -no-pie 关闭 PIE，使地址在当前编译环境中保持固定，便于重复演示。

三、六层架构

层	目录	行数	关键类
① 基础设施	src/core/	665	BinaryFile / MappedBinaryFile · MemoryArena · SimpleVector<T> · Log
② ELF 解析	src/elf/	1714	ElfParser · ElfHeaderView · SectionTable · StringTable · SymbolTable · FunctionTable
③ 指令解码	src/arch/	1627	InstructionDecoder · X86_64Decoder · X86OpcodeTable · InstructionStream
④ CFG	src/cfg/	818	ControlFlowGraph · BasicBlockBuilder · EdgeAnalyzer · BasicBlock · Edge
⑤ 分析	src/analysis/	494	AnalysisPass · ReachabilityPass · LoopHintPass · StatisticsPass
⑥ 输出	src/output/	353	GraphWriter · DotGraphWriter · TextWriter
入口	src/main.cpp	321	命令行解析与流程编排

合计 **49** 个源文件、**5992** 行。依赖方向严格单向：① ← ② ← ③ ← ④ ← ⑤/⑥。上层只通过**抽象基类指针**与下层交互，因此每一层都可以单独替换或测试。

四、ELF 解析模块（自己实现）

只使用 Linux 系统头文件 <elf.h> 的结构体定义

Elf64_Ehdr / Elf64_Shdr / Elf64_Sym 等只是 ELF 格式的数据布局描述，不是解析库。解析逻辑全部自己写。

4.1 文件访问：open → fstat → mmap

```
int fd = open(path, O_RDONLY);           /* 打开          */
fstat(fd, &st);                          /* 取大小        */
void *m = mmap(NULL, st.st_size, PROT_READ, MAP_PRIVATE, fd, 0); /* 只读映射      */
... 分析全程只在映射区上进行 ...
munmap(m, size);  close(fd);            /* 析构时释放    */
```

选择 mmap 的原因：ELF 分析是"拿着偏移直接取字节"的场景，映射后 data() + offset 就是指针，零拷贝；且 data() 与 size() 构成完整可访问区间，所有边界检查都基于这两个值。

4.2 文件头六项校验

检查项	失败时的报错示例
长度 ≥ sizeof(Elf64_Ehdr)	文件过小：40 字节，不足以容纳 ELF64 文件头（64 字节）
magic = 7f 45 4c 46	不是 ELF 文件：magic 为 50 4b 03 04，应为 7f 45 4c 46
EI_CLASS == ELFCLASS64	不是 ELF64（EI_CLASS = 1）
EI_DATA == ELFDATA2LSB	不是小端 ELF（EI_DATA = 2）
EI_VERSION == EV_CURRENT	不支持的 ELF 版本
e_machine == EM_X86_64	不是 x86-64 目标文件（e_machine = 183）

4.3 函数定位：从符号到机器码

```
st_value (虚拟地址，来自符号表)
| 找到 st_shndx 对应的节: secAddr = sh_addr, secOff = sh_offset
▼
offsetInSection = st_value - secAddr      ← 节内偏移
fileOffset      = secOff + offsetInSection ← 文件偏移
code            = fileData + offsetInSection ← 机器码首字节指针
```

以 classify 为例（实测）：st_value = 0x4011cd，.text 节 sh_addr = 0x401050、sh_offset = 0x1050，故 fileOffset = 0x1050 + 0x17d = 0x11cd，机器码内容为 55 48 89 e5 ...（即 push %rbp; mov %rsp,%rbp），大小 81 字节。

4.4 四类边界情况都有处理

情况	处理方式
符号未定义 / 特殊索引	SHN_UNDEF、SHN_ABS 等直接视为"没有机器码"
st_size == 0	GCC 的 __init 等常见；用"同节内下一个函数符号"或"节末尾"兜底推算
st_size 超出节范围	截断到节内剩余空间并告警
机器码范围超出文件	最后防线：拒绝并置 code = NULL

► 终端同步演示 ./elfcfg --info demo ./elfcfg --sections demo ./elfcfg --symbols demo

五、x86-64 解码器（本次升级的核心）

架构相关与架构无关的分界线

`InstructionDecoder` 是抽象基类，`X86_64Decoder` 是派生类。上层（`InstructionStream`、CFG 各模块）只认识基类指针，完全不知道底下是什么指令集。更换目标架构只需新增一个派生类，上层代码一行不改。

5.1 指令格式与解码七步

Legacy Prefix	REX	Opcode	ModRM	SIB	Disp	Imm
(0 或多个)	(0/1)	(1/2/3 字节)	(0/1)	(0/1)	(0/1/4)	(0..8)

1. **Legacy 前缀**：LOCK `F0`、REP/REPNE `F2/F3`、段前缀 `2E 36 3E 26 64 65`、`66`、`67`（可重复、顺序任意）
2. **REX**：`40 - 4F`，解析 W/R/X/B 四个扩展位
3. **Opcode**：单字节，或 `0F xx` 两字节；`0F 38 / 0F 3A` 三字节明确报错
4. **ModRM**：拆出 mod / reg / r/m
5. **SIB**：仅当 `mod≠3 且 r/m==4` 时存在，拆出 scale / index / base
6. **位移**：由 mod 与 r/m 共同决定长度（见下表）
7. **立即数**：由 opcode 与 REX.W / `66` 共同决定长度

5.2 长度是"推导"出来的，不是猜的

条件	位移长度
<code>mod == 0 且 r/m == 5</code>	disp32（RIP 相对寻址）
<code>mod == 0</code> 、有 SIB 且 <code>base == 5</code>	disp32（无基址）
<code>mod == 1</code>	disp8
<code>mod == 2</code>	disp32
<code>mod == 3</code>	无位移（寄存器直接寻址）

两个易错点：`SIB.index == 4` 表示"没有变址"（不是 `rsp`）；`SIB.base == 5 且 mod == 0` 表示"没有基址"（不是 `rbp`）。`0x83` 与 `0x81` 都是 Group1，但前者用 `imm8` 并符号扩展。

5.3 关键约束：绝不猜测未知指令的长度

x86 是变长指令集，长度算错一条，后面全部错位。因此遇到不支持的 opcode 时，解码器立即返回失败，`InstructionStream` 停止该函数的解码并报出具体地址与 opcode 字节：

`[elfcfg] ERROR: 解码停止于 0x4012ab (函数内偏移 53) : 不支持的 opcode 0F 28`

已解出的部分仍然保留，可用于分析。绝不会把未知指令当作 1 字节继续解码。

5.4 操作数顺序：本项目踩过的坑

Intel 语法与 AT&T 语法的操作数顺序是相反的，本项目输出 AT&T 风格（带 % 与 \$ ），因此需要交换：

机器码	Intel 语法	本项目输出（AT&T）
48 89 e5	mov rbp, rsp	mov %rsp,%rbp
8b 45 ec	mov eax, [rbp-0x14]	mov -0x14(%rbp),%eax
89 45 fc	mov [rbp-0x4], eax	mov %eax,-0x4(%rbp)

最初实现把顺序写反了，靠与 objdump 逐条比对才发现 —— 这也是保留交叉验证脚本的原因。

5.5 支持的控制流指令（CFG 依赖，全部支持）

指令	编码
短条件跳转 Jcc（16 种条件）	70–7F rel8
近条件跳转 Jcc（16 种条件）	0F 80–0F 8F rel32
无条件跳转 JMP	EB rel8 / E9 rel32
函数调用 CALL	E8 rel32
返回 RET	C3 / C2 imm16
循环 LOOP / LOOPE / LOOPNE / JRCXZ	E2 / E1 / E0 / E3 rel8
间接 CALL / 间接 JMP	FF /2 · FF /4（目标标记为不可知）

数据传送与运算还支持：mov / lea / movzx / movsx / movsxd / push / pop / add / or / adc / sbb / and / sub / xor / cmp / test / not / neg / mul / imul / div / idiv / inc / dec / 移位 / xchg / cmpxchg / xadd / nop / leave / cwde / cdq / cmovcc / setcc / bswap / syscall / cpuid / rdtsc 等。

► 终端同步演示 `./elfcfg --disasm demo classify` —— 这是我们自己解码的结果，不是 objdump

六、基本块识别模块

BasicBlockBuilder (抽象基类)

➡

LinearBasicBlockBuilder (派生类)

算法：Leader (入口) 法，与早期版本一致。

规则	内容
规则 1	函数的第一条指令是一个基本块的入口
规则 2	跳转指令的目标地址是一个基本块的入口
规则 3	跳转指令之后的下一条指令也是一个基本块的入口
补充	ret / hlt 之后也另起一块，避免把不可达代码并进上一块

多态在这里的作用

```
/* 基类：只有跳转才结束基本块 */
int BasicBlockBuilder::endsBlock(const Instruction *ins) const {
    return (ins->type == INST_JMP || ins->type == INST_JCC ||
            ins->type == INST_LOOP) ? 1 : 0;
}
/* 派生类：x86 上 ret / hlt 也结束基本块 — 划分结果因此不同 */
int LinearBasicBlockBuilder::endsBlock(const Instruction *ins) const {
    if (BasicBlockBuilder::endsBlock(ins)) return 1;
    return (ins->type == INST_RET || ins->type == INST_HLT) ? 1 : 0;
}
```

本例中：26 条指令 → 9 个基本块 (B0 ~ B8)

七、基本块执行关系识别模块

EdgeAnalyzer (抽象基类) ➡ X86EdgeAnalyzer (派生类)

核心观察：一个基本块的全部出边，只由它的最后一条指令决定。

块尾指令	出边	EdgeType 与目标
条件跳转 jcc / LOOP	2 条	EDGE_BRANCH_TRUE (跳转目标) + EDGE_BRANCH_FALSE (顺序下一块)
无条件跳转 JMP (直接)	1 条	EDGE_UNCONDITIONAL (不再顺序执行)
无条件跳转 JMP (间接)	1 条	EDGE_INDIRECT (目标不可静态确定, to = NULL)
RET / HLT	0 条	函数出口, 控制流终止
块尾是 CALL	1 条	EDGE_CALL (返回后继续的下一块)
其它 (含块内 call)	1 条	EDGE_FALLTHROUGH

关于图中的红色加粗边：判定方式是"目标块地址 ≤ 源块地址"，说明控制流往回跳。

需要说明的是，严格意义的 CFG back edge 需要用支配关系判定（目标块支配源块），本实现未构造支配树，因此这是向后跳转边的演示性近似，代码与文档中统一称为 backward jump hint。在 classify 中，这条 B7 → B6 确实对应源码的 for 循环。

本例中：9 个基本块 → 11 条基本块执行关系（其中 1 条为向后跳转边）

八、CFG 与分析 Pass

8.1 ControlFlowGraph —— CFG 层的中心对象

```
class ControlFlowGraph {
    BasicBlock *addBlock(uint64_t startAddress, Instruction *first, Instruction *last);
    Edge      *addEdge(BasicBlock *from, BasicBlock *to, EdgeType type,
                      uint64_t targetAddress, bool targetResolved, const char *label);
    BasicBlock *findBlockByAddress(uint64_t address) const; /* 入口地址精确匹配 */
    BasicBlock *blockContaining(uint64_t address) const;    /* 地址落在区间内 */
    size_t     blockCount() / edgeCount() / instructionCount();
    ...
};
```

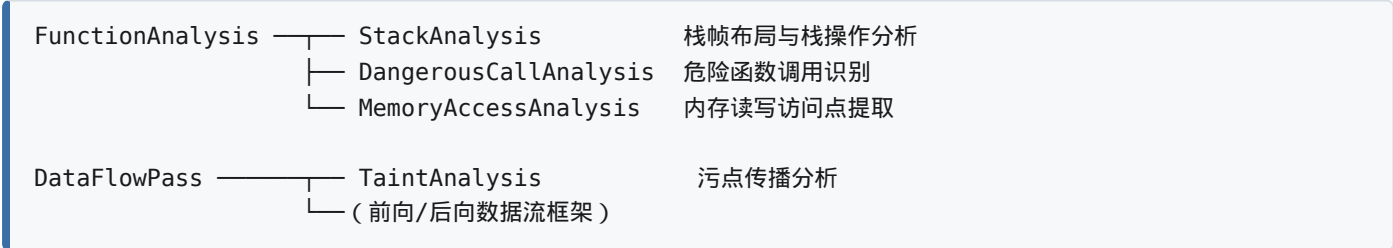
块与边全部由 **MemoryArena** 分配（64 KiB 分块、块内线性切分、分配即清零），程序退出时一次性归还 —— 不存在“漏 free 某条链表”的问题，也没有几万次 malloc 的开销。

8.2 分析 Pass：新增分析不必改主流程

```
AnalysisPass *passes[3]; /* 基类指针数组统一驱动 —— 运行时多态 */
passes[0] = &reachability; /* 可达性 */
passes[1] = &loopHint; /* 循环提示 */
passes[2] = &statistics; /* 统计 */
for (int i = 0; i < 3; ++i) passes[i]->run(&cfg);
```

Pass	作用	输出
ReachabilityPass	从入口块 BFS，标记可达基本块	BasicBlock::reachable
LoopHintPass	识别向后跳转边（演示性近似）	BasicBlock::loopHeader
StatisticsPass	指令/块/边/分支/call/ret/间接跳转统计	StatisticsPass::Stats

8.3 为后续实验预留的接口（骨架已就位）



目前是抽象接口，尚未实现具体算法。目的是让后续实验可以“新增一个派生类”即可接入，而不必推翻现有结构。

九、输出层

GraphWriter (抽象基类) → DotGraphWriter / TextWriter

输出层只依赖 `ControlFlowGraph`，不关心它是怎么来的。同一个 CFG 既能输出成 Graphviz DOT (交给外部 dot 渲染)，也能输出成人类可读的文本。

- **节点**：每个基本块一个方框，框内**逐行列出该块的每条指令**（地址 + 助记符 + 操作数）
- **边**：每条执行关系一个箭头，标签为触发该分支的指令（`jle` / `jmp` / `fall`）
- **配色**：入口块绿色、以 `ret` 结尾的出口块红色、向后跳转边红色加粗、不可达块灰色虚线框
- **间接跳转**：画成虚线的悬挂边，明确表示"目标不可静态确定"

程序不依赖 **Graphviz 库**：elfcfg 把 DOT 写到 stdout 后即运行结束；`dot -Tpng` 属于**程序外部**的展示步骤，不参与任何分析。

► **终端同步演示** `./elfcfg demo classify > classify.dot` 然后 `dot -Tpng classify.dot -o classify.png`

十、面向对象设计：继承与多态

10.1 六组继承体系

抽象基类	派生类	纯虚 / 虚函数	多态的意义
BinaryFile	MappedBinaryFile	openFile / data / size	换 read() 实现时上层不变
InstructionDecoder	X86_64Decoder	decode()	换架构只需换派生类
BasicBlockBuilder	LinearBasicBlockBuilder	build() + endsBlock()	不同划分策略可替换
EdgeAnalyzer	X86EdgeAnalyzer	analyze() + analyzeBlock()	不同指令集出边规则可替换
AnalysisPass	Reachability / LoopHint / Statistics	run()	新增分析不必改主流程
GraphWriter	DotGraphWriter / TextWriter	write()	可扩展 JSON 等输出格式

10.2 main 中的实际调用

```
BinaryFile      *file      = new MappedBinaryFile();          /* 由 ElfParser 内部持有 */
InstructionDecoder *decoder = new X86_64Decoder();
BasicBlockBuilder *builder = new LinearBasicBlockBuilder();
EdgeAnalyzer     *analyzer= new X86EdgeAnalyzer();
GraphWriter      *writer   = new DotGraphWriter();

AnalysisPass *passes[3];                                     /* 基类指针数组统一驱动 */
passes[0] = &reachability;
passes[1] = &loopHint;
passes[2] = &statistics;
```

- 多态不是形式上的：
- ① `InstructionStream` 只认识 `InstructionDecoder*`，换架构只需换派生类；
 - ② `endsBlock()` 被派生类重写后，`ret` 也成了基本块的结束条件，划分结果因此不同；
 - ③ `analyzeBlock()` 基类只连顺序边，派生类加入 x86 跳转语义；
 - ④ `main` 遍历 `AnalysisPass*` 数组，新增分析 pass 不必改主流程；
 - ⑤ 所有基类都声明了虚析构函数，保证 `delete 基类指针` 时资源正确释放。

十一、编译与运行结果

11.1 手工编译命令

```
gcc -x c++ -std=c++11 -O0 -g -Wall -Wextra -Wpedantic -I src \
src/main.cpp src/core/*.cpp src/elf/*.cpp src/arch/*.cpp src/arch/x86/*.cpp \
src/cfg/*.cpp src/analysis/*.cpp src/output/*.cpp -lstdc++ -o elfcfg
```

选项	说明
-x c++ -std=c++11	用 gcc 手工按 C++11 编译
-Wall -Wextra -Wpedantic	打开全部警告；本程序编译结果 0 错误 0 警告
-O0 -g	保留调试信息，便于后续 gdb 实验；不 strip
-lstdc++	手工链接 C++ 标准库（未使用任何 STL 容器 ）

```
► 终端同步演示 gcc -x c++ -std=c++11 -O0 -g -Wall -Wextra -Wpedantic -I src src/main.cpp
src/core/*.cpp src/elf/*.cpp src/arch/*.cpp src/arch/x86/*.cpp src/cfg/*.cpp
src/analysis/*.cpp src/output/*.cpp -lstdc++ -o elfcfg
```

11.2 六个测试函数的实测数据

函数	地址	字节	指令	基本块	边	向后跳转边	说明
simple_if	0x401136	28	11	4	4	0	最简单的条件分支
nested_if	0x401152	65	18	7	9	0	嵌套条件
loop_sum	0x401193	59	19	7	8	2	for + while 两条循环
classify	0x4011cd	81	26	9	11	1	课堂主用例
switch_test	0x40121e	68	21	9	9	0	跳转表，见已知限制
call_test	0x401262	65	23	1	0	0	无分支，顺序执行到 ret

```
► 终端同步演示 ./elfcfg --summary demo classify —— 统计、循环提示、基本块列表一次看清
```

11.3 控制流图结果

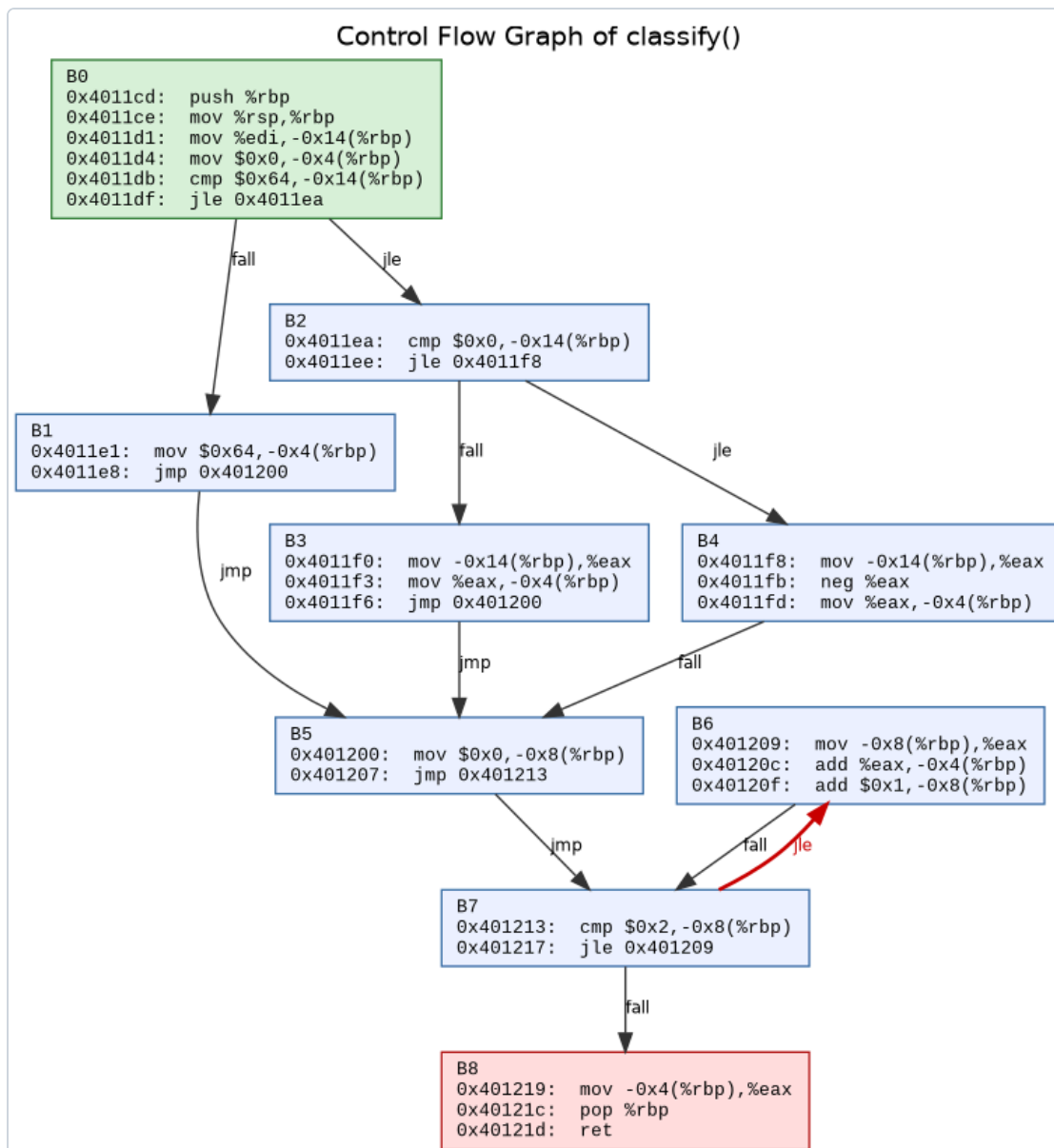


图 1 classify 函数的控制流图（绿色 B0 = 入口，红色 B8 = 出口，红色加粗 B7→B6 = 向后跳转边）

11.4 源码 ↔ 解码结果 ↔ CFG 三方对应

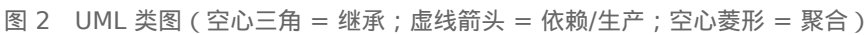
源码语句	基本块（地址区间）	CFG 中的表现
序言、 <code>sum=0</code> 、 <code>if (x>100)</code>	B0 0x4011cd - 0x4011e1	块尾 <code>jle</code> ，产生两条出边
<code>sum = 100;</code>	B1 0x4011e1 - 0x4011ea	块尾 <code>jmp</code> ，无 fall 边
<code>else if (x > 0)</code>	B2 0x4011ea - 0x4011f0	块尾 <code>jle</code> ，两条出边
<code>sum = x;</code>	B3 0x4011f0 - 0x4011f8	块尾 <code>jmp</code>
<code>sum = -x;</code>	B4 0x4011f8 - 0x401200	顺序落入 B5
分支汇合、 <code>for</code> 初始化	B5 0x401200 - 0x401209	块尾 <code>jmp</code>
循环体 <code>sum += i; i++</code>	B6 0x401209 - 0x401213	循环体
循环条件 <code>i < 3</code>	B7 0x401213 - 0x401219	块尾 <code>jle</code> ，向后跳转边来源
<code>return sum;</code>	B8 0x401219 - 0x40121e	块尾 <code>ret</code> ，无出边

读图提醒：汇编中的 `jle` 含义是“小于等于就跳走”，因此对条件跳转而言，跳转边对应的是源码条件“不成立”，fall 边才对应源码条件“成立”——两者方向相反。

11.5 交叉验证（开发期 oracle，程序不调用）

验证方式	结果
解码结果 vs GNU objdump（逐条比对）	classify 26/26 条、main 26/26 条 —— 地址、助记符、操作数全部一致
CFG 结构 vs 早期 objdump 版本（逐边比对）	11 条边的连接关系与标签完全一致
节表 vs <code>readelf -S</code>	逐项一致 （如 <code>.interp</code> 地址 0x400318、偏移 0x318、大小 28 字节）

验证脚本位于 `tests/verify_against_objdump.py`，仅由人工在开发/验收阶段运行。这是“用外部工具做 oracle”的用法，程序源码中不存在任何对外部工具的调用。



output。三个数据结构 `Instruction`、`BasicBlock`、`Edge` 被各层共用，构成贯穿全程的数据流。

十三、程序流程图

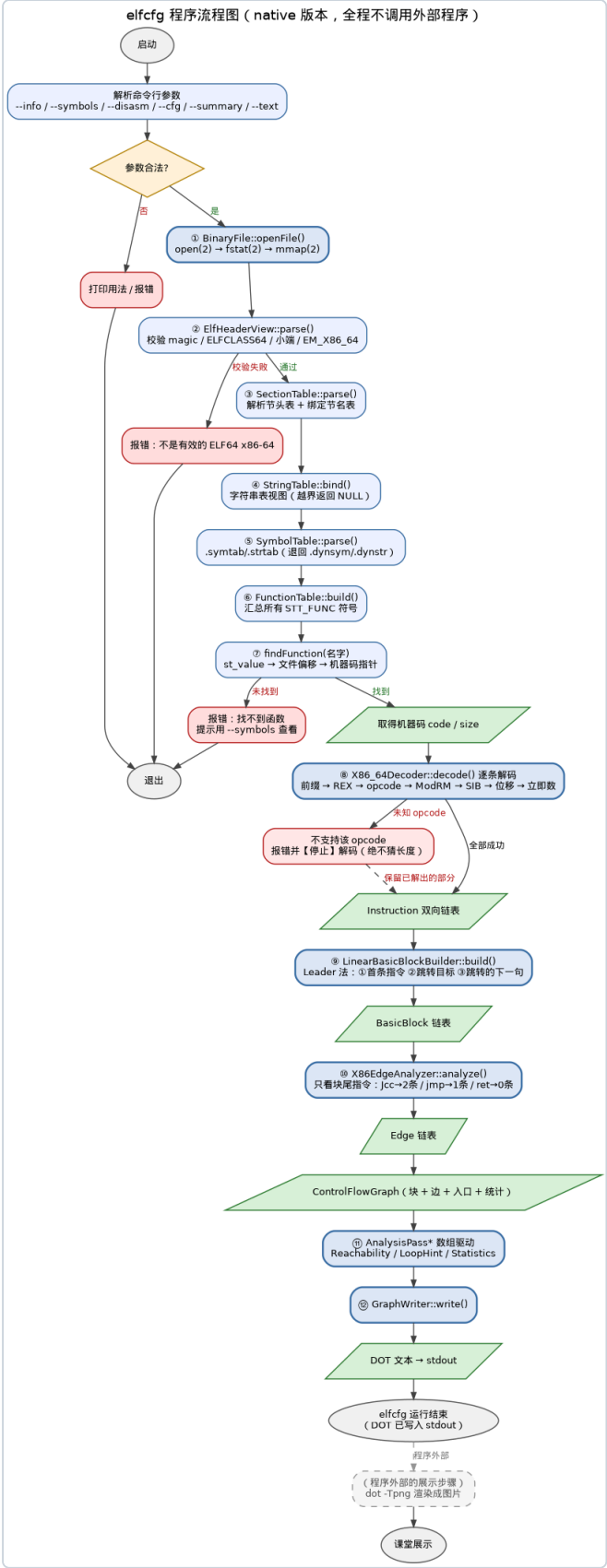


图 3 程序流程图 (虚线框内为程序外部的展示步骤)

流程中的两个硬约束：

- ① 遇到不认识的 **opcode** 立即停止解码并报错 —— 绝不假定它长 1 字节继续，否则后续全部错位；
- ② **dot -Tpng** 属于程序外部的展示步骤，elfcfg 输出 DOT 到 stdout 后即运行结束。

十四、已知限制（如实记录）

#	限制	现象 / 影响
1	解码器非完整实现	不支持三字节 opcode (<code>0F 38</code> / <code>0F 3A</code>)、SSE/AVX、x87 浮点、位操作指令；遇到即报错停止
2	间接跳转目标不可解析	<code>jmp %rax</code> 生成 <code>EDGE_INDIRECT</code> 边，目标块为空，可达性分析会漏掉经由该边到达的块
3	switch 跳转表	能识别为间接跳转，但不读跳转表内容，各 case 对应的块会被误判为不可达（见 <code>--text demo switch_test</code> ）
4	"向后跳转边"是演示性近似	判据为"目标地址 $\leq$ 源地址"， 不是 严格 back edge（严格需支配关系）；统一称 backward jump hint
5	依赖符号表	strip 后的二进制无法按名字定位函数，会明确报错并提示用 <code>--symbols</code> 查看可用函数
6	其它	只支持 ELF64 小端 x86-64；只分析函数内部 CFG；不解析 <code>.eh_frame</code> ；不处理重定位

14.1 后续可扩展的五个方向

#	实验	接入点
1	支配树与真实回边	<code>analysis/LoopHintPass</code> 的严格版本，实现支配树后替换地址比较判据
2	栈帧与栈溢出分析	<code>analysis/StackAnalysis</code> （骨架已声明）； <code>Instruction::fields</code> 保留了 ModRM/SIB/dispatch
3	危险函数调用识别	<code>analysis/DangerousCallAnalysis</code> ；需让 <code>call rel32</code> 的目标可反查符号
4	污点 / 数据流分析	<code>analysis/DataFlowPass</code> + <code>TaintAnalysis</code> ，前向/后向框架与 <code>solve()</code> 接口已定义
5	跳转表还原	扩展 <code>X86EdgeAnalyzer</code> 处理 <code>FF /4</code> ，读只读节并校验表项合法性

14.2 现实意义

控制流图是软件安全领域的基础分析手段：**恶意代码分析**中用它理解样本行为与去混淆；**模糊测试**（如 AFL）用边覆盖率引导输入变异；**编译器优化**的数据流分析建立在 CFG 之上；**控制流完整性（CFI）**防护通过校验运行时控制流是否符合编译期算出的 CFG，来抵御 ROP 等攻击。本练习实现的正是这条技术链路的第 0 步——而且是从 ELF 字节流开始的第一步。